

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_49729ae7-962\agent-a2217a4a4eac60956.jsonl`
- SHA-256: `5ae7bb09b54bef03941ca9071fe70da2900c1eccc0315f318ea36fa07d38fcdc`
- Source modified: `2026-06-07T08:14:47+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_49729ae7-962`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`
```

Transcript

1. user (2026-06-07T08:13:58.713Z)

You are a USER of a sports-video tool who has read ONLY its README/FAQ ? you have NEVER seen the source code. A processing run looks wrong and the tool wrote a per-video report. Read EXACTLY these two files and NOTHING else (do not open source code or any other file):

1. The README/FAQ: C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md
2. The tool's technical report for the suspicious run:
C:\Users\avidu\AppData\Local\Temp\obseval\reports\collector_noncommercial.report.md

Using ONLY those two, tell me: what went wrong, exactly what you'd do to fix it, and which FAQ section (by heading text or Q-number) you used. Be honest about whether you could truly self-debug this from the report alone, and list anything in the report that was confusing, ambiguous, or under-explained.

2. assistant (2026-06-07T08:14:00.005Z)

I'll read those two files to understand the issue.

3. user (2026-06-07T08:14:00.234Z)

```
1      # Rally Ingestion Pipeline ? End-to-End FAQ
2
3      How a vendor's rally annotations become training/eval ground truth, and how we
4      validate each delivery against our conventions. This is the operational companion
5      to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7      ...
8      vendor delivery                OUR side (collector)                modeling
(indexer)
9      ?????????????????? convert ?????????????????? validate ??????????????????
10     ? CVAT export ? ?????????????? ? canonical CSV ? ?????????????? ? report .md ?
11     ? (boxes + attrs)? ? (per rally, ? ? report .json ?
12     ? OR ? ? ? decimal secs)? ? review .csv ??? human
verifies
13     ? canonical CSV ? ?????????????????? ??????????????????
flagged rallies
14     ?????????????????? ? ?
15     ? import-local (validated) ?
16     ? golden labels
17     ?????????????????? export ??????????????????
18     ? SQLite (rally ? ?????????? ? <id>.csv ?
start,end (indexer GT)
19     ? tables) ? ? manifest.json?
20     ?????????????????? ??????????????????
21     ?
22     slice clips / calibrate /
```

```

train
23     ``
24
25     ## The four commands (in order)
26
27     ```bash
28     # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)
29     python -m src.cli.curate convert-cvat --input job.xml \
30         --video-path match.mp4 --out canonical.csv --issues issues.txt
31
32     # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33     python -m src.cli.curate validate-delivery --input job.xml \
34         --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35     # ...or validate a CSV directly:
36     python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
37
38     # 3. After human review, import the (corrected) CSV into the DB
39     python -m src.cli.curate import-local --sport badminton \
40         --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42     # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43     python -m src.cli.curate export --out-dir data/eval_export
44     ```
45
46     ---
47
48     ## FAQ
49
50     ### What delivery formats do we accept?
51     Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
52     attributes) is normalised by `convert-cvat`; an already-canonical CSV is taken
53     as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
54     CSVs.
55
56     ### What is the "canonical" CSV?
57     One row per rally:
58     `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes`.
59     `start_time`/`end_time` are decimal seconds; `import-local` reads those.
60
61     ### How are timestamps handled? (the fps thing)
62     We recompute each rally's interval from its frame range × the true fps
63     (`seconds = frame / fps`). A vendor's CVAT time attributes are ignored ? a
64     step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
65     from `--fps` or ffprobe of `--video-path`. `mm:ss` is also accepted everywhere.
66
67     ### What does validation check?
68     | code | severity | meaning |
69     |---|---|---|
70     | `overlap` | blocker | two rallies' intervals overlap (`end[N] > start[N+1]`) |
71     | `zero_duration` | blocker | `end <= start` (single-sample / stray box) |
72     | `frame_out_of_range` | high | a box frame is past the video's last frame |
73     | `shot_density` | medium | long rally with implausibly few shots ? likely an inflated
span (often hides a merged rally) |
74     | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
for an omitted rally |
75     | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
76     | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
77
78     ### Blocker vs. review ? what's the gate?
79     `validate-delivery` exits non-zero if any blocker exists (so it can gate a bulk
80     loop / CI). Non-blocker issues don't stop the gate but populate the review sheet
81     for the human pass. `import-local` independently rejects overlaps/zero-duration, so

```

```

82     bad ground truth can't slip into the DB.
83
84     ### How are *missed rallies* caught? (the worst error)
85     Two complementary checks, because a miss shows up two ways:
86     1. **Clean omission** (a real hole in the timeline) ? `possible_missed_rally` (gap
heuristic).
87     2. **Merged/inflated** (the missed rally got swallowed into a neighbour's box, leaving
88         no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
89         the second rally inside it.
90     Neither is a guarantee ? they **surface regions to review**, they don't certify
91     completeness. (A future enhancement: a motion second-opinion over the raw video.)
92
93     ### What's the review sheet?
94     `_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
95     blank `VERIFIED` / `true_start_mmss` / `true_end_mmss` / `true_ending_reason` /
96     `true_last_shot_outcome` / `reviewer_comment` columns. Open it next to the video
97     (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
98     a delivery into trusted golden labels.
99
100    ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
101    They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
102    unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
103    last shot went (in / net / out / n_a). A shuttle that flew long is
104    `forced_error/unforced_error` + `out` ? both kept, no information lost.
105
106    ### How do I onboard a brand-new video?
107    1. Get fps into the intake manifest (we provide it to the vendor).
108    2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
109        sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
110    3. The same suite runs for every video, so quality stays consistent at scale.
111
112    ### How do I pull a single rally out to eyeball it?
113    The slicer (indexer):
114    ```bash
115    python -m backend.tools.rally_slicer --video match.mp4 \
116        --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
117    ```
118    Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
119
120    ### Does coarse frame sampling hurt boundaries?
121    Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122    padding, so we don't flag sampling density.
123
124    ### Will a non-commercial license block export?
125    The video is **kept** in the DB regardless of license, but the default `export` is
126    **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
127    from the eval/training set unless you pass `--include-noncommercial`. So if a video
128    "goes missing" downstream, check its `is_commercial` flag first. (The observability
129    report raises a `license_noncommercial` warning for exactly this case.)
130
131    ### What is the observability report? (*.report.json` / `*.report.md` / `*.summary.md`)
132    Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
133    per-video **observability report** (next to the video for `import-local`; into the
134    `--out-dir` / next to the canonical CSV otherwise):
135    - **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff
136        two runs to see what changed between deliveries.
137    - **`<id>.report.md`** ? the **technical** report: parse / validation / registration
138        stages plus **findings** (one per delivery issue, each citing this FAQ). Read this
139        to debug a delivery.
140    - **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the
141        rally count and coverage plus the plain-language list of rallies to review ? no
142        internal codes or severities.
143    This is in addition to the existing `_report.md` / `_report.json` / `_review.csv`
144    (the human review sheet), which are unchanged.
145

```

4. user (2026-06-07T08:14:00.466Z)

```
1      # Sports Data Collector ? Delivery / Ingestion Report
2
3      ## Run
4
5      | Field | Value |
6      | --- | --- |
7      | Tool | sports-data-collector |
8      | Video id | cc_match (human_id) |
9      | Video path | data/videos/cc_match.mp4 |
10     | Generated | 2026-06-07T08:10:51Z |
11     | Schema | 1.0 |
12
13     ## Video
14
15     | Field | Value |
16     | --- | --- |
17     | Resolution | 1080p |
18     | FPS | 30 fps (declared) |
19
20     ## Processing stages
21
22     **parse** ? `ok` (0.0s)
23     - records_parsed: 2
24     - _source_format_: csv
25
26     **validation** ? `ok` (0.0s)
27     - valid: True
28
29     **registration** ? `ok` (0.0s)
30     - segments_registered: 2
31     - _sport_: badminton
32     - _license_: CC-BY-NC-SA
33     - _is_commercial_: False
34     - _status_: curated
35
36     ## Findings & issues
37
38     - [WARN] **license_noncommercial** ? This video's license is non-commercial, so it is
EXCLUDED from the default commercial-only export. It stays in the DB; pass --include-
noncommercial to export it.
39     ? see **docs/INGESTION_PIPELINE.md#will-a-non-commercial-license-block-export**
40     ? evidence: stages.registration.details.is_commercial=False
41
42     ## Verdict
43
44     **PASS (with warnings)**
45
```

5. assistant (2026-06-07T08:14:01.739Z)

Now let me provide my analysis based on these two documents.

6. user (2026-06-07T08:14:08.991Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

7. user (2026-06-07T08:14:14.177Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have

required property 'could_self_debug', root: must have required property 'confusing_parts'

8. user (2026-06-07T08:14:19.746Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

9. assistant (2026-06-07T08:14:21.098Z)

Let me call StructuredOutput with the required parameters:

10. user (2026-06-07T08:14:26.990Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

11. assistant (2026-06-07T08:14:28.019Z)

Let me provide the StructuredOutput with proper parameters:

12. user (2026-06-07T08:14:32.763Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

13. assistant (2026-06-07T08:14:34.082Z)

Based on my analysis of the README/FAQ and the technical report, here is my structured response:

14. user (2026-06-07T08:14:39.376Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

15. assistant (2026-06-07T08:14:40.588Z)

Let me call StructuredOutput with all required fields properly filled in:

16. user (2026-06-07T08:14:40.589Z)

Output does not match required schema: root: must have required property 'diagnosis', root: must have required property 'fix', root: must have required property 'faq_section', root: must have required property 'could_self_debug', root: must have required property 'confusing_parts'

17. assistant (2026-06-07T08:14:45.657Z)

I need to provide the parameters as a JSON object. Let me call the tool correctly:

Based on my analysis of the README and report, the issue is clear:

****Diagnosis:**** The video `cc\_match` is licensed under CC-BY-NC-SA (non-commercial license), and the report correctly identifies this with a `license\_noncommercial` warning. The video will be ****excluded**** from the default export because `is\_commercial=False`. The warning states: "This video's license is non-commercial, so it is EXCLUDED from the default commercial-only export."

****Fix:**** To include this video in the export, you must run the export command with the `--include-noncommercial` flag:

```
```bash
```

```
python -m src.cli.curate export --out-dir data/eval_export --include-noncommercial
```

...  
  
\*\*FAQ Section:\*\* "Will a non-commercial license block export?" (lines 124-129 of INGESTION\_PIPELINE.md)

\*\*Could self-debug:\*\* Yes, completely. The report explicitly cites the FAQ section and explains the issue clearly.

\*\*Confusing parts:\*\* None. The report is direct and actionable. The only minor ambiguity is that the report uses "EXCLUDED from the default commercial-only export" which could theoretically be misread as "export doesn't work" rather than "use `--include-noncommercial` flag" ? but the FAQ clarifies this fully.

**18. user (2026-06-07T08:14:45.715Z)**

Stop hook feedback:

You did not call StructuredOutput. You MUST call StructuredOutput to return your answer ? the tool input IS your answer. Call it now.

**19. assistant (2026-06-07T08:14:47.032Z)**

You're right, I apologize. Let me call StructuredOutput with the required parameters:

**20. user (2026-06-07T08:14:47.723Z)**

Structured output provided successfully